

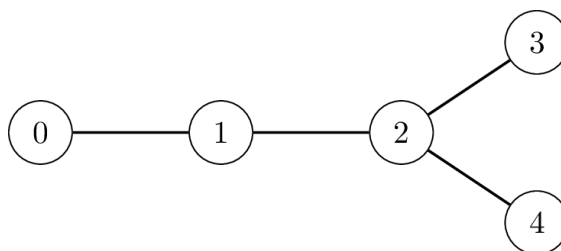
Migrationer

Harry undersöker migrationsmönstrena av dinosaurier i Bolivia. Han har undersökt dinosauriefotavtryck på N olika ställen, numrerade från 0 till $N - 1$ i minskade ordning av gammelhett: ställe 0 har de äldsta fotavtrycken, medan ställe $N - 1$ har de nyaste.

Dinosaurierna migrerade till varje ställe (förutom ställe 0) från ett visst äldre ställe. För varje ställe i där $1 \leq i \leq N - 1$, så finns det exakt ett äldre ställe $P[i]$ (där $P[i] < i$) så att några dinosaurier migrerade från ställe $P[i]$ till ställe i . Ett givet äldre ställe kan vara källan av migreringar till flera nyare ställen.

Harry modellera varje migrering som en **orientad sammankoppling** mellan ställe i och $P[i]$. Notera att för två olika ställen x och y , så är det alltid möjligt att resa från x till y genom att färdas längs med en sekvens av sammankopplingar. **Avståndet** mellan ställe x och y är definierat som minsta antalet sammankoppling man behöver korsa för att resa från x till y .

Exempelvis: följande bild visualiserar ett fall med $N = 5$ ställen, där $P[1] = 0$, $P[2] = 1$, $P[3] = 2$, och $P[4] = 2$. Man kan resa från ställe 3 till ställe 4 med hjälp av 2 sammankoppling, så avståndet mellan de är 2.



Eftersom Harry gillar långa resor vill han hitta ett par ställen med största möjliga avstånd.

Notera att ett par med denna egenskap inte behöver vara unik: i exemplet ovanför har både par $(0, 3)$ och $(0, 4)$ med ett avstånd av 3, vilket är det största möjliga. I sådana fall kan du ge vilket par som helst som maximerar avståndet.

Tyvärr lider Harry av personlighetsstörning (🧠), och har därför två möjliga tillstånd: inlåst 🧠 och utelåst 🧠. När Harry är hemma är han inlåst 🧠, och varje gång han lämnar huset blir han utelåst 🧠. Dessa två tillstånd **delar inga minnen** mellan varandra. Han vill nu lämna huset, (och då vara

utelåst 🐼), samla information och skicka tillbaka brev så att han kan avgöra ett par ställen med största möjliga avstånd när han återvänder hem och blir inlåst 🐼 igen.

Till en början är alla värden av $P[i]$ **okända**. Harry kommer sedan besöka ställe $1, 2, \dots, N - 1$ i denna ordningen. Han är just nu uppenbarligen utelåst 🐼. När han besöker varje ställe i ($1 \leq i \leq N - 1$), så utför han följande hand(👉)lingar:

- Först avgör han värdet av $P[i]$, det vill säga, källan av migrationen till ställe i .
- Sedan avgör han vare sig han vill skicka **ett** brev till sitt hus eller inte. För att fatta ett väl informerat beslut har han tillgång till tidigare insamlad information.

Eftersom han skickar meddelandena med brev, så måste varje brev innehålla ett heltal mellan 1 och 20 000, inklusive (postnord accepterar inte större tal). Han kan skicka **som mest 50 brev**, eftersom han inte har så många frimärken.

Du ska nu formulera en strategi, sådan att:

- Harrys utelåsta 🐼 personlighet väljer vilka ställen han skickar brev ifrån, och exakt vad han skickar från ställena han skickar brev ifrån.
- När Harry kommer hem och blir inlåst 🐼, så kan han endast använda informationen som står i breven för att avgöra ett par ställen med största möjliga avstånd.

Det är billigare att skicka brev med lägre tal. Därmed kommer antalet poäng du får bero både på det största talet du skickar och totala antalet brev du skickar.

Implementationsdetaljer

Du behöver implementera två procedurer; en för när Harry är inlåst 🐼, och en när han är utelåst 🐼.

När Harry är utelåst 🐼 ska du implementera följande procedur:

```
int send_message(int N, int i, int Pi)
```

- N : antalet ställen med fotavtryck.
- i : indexet av stället som Harrys utelåsta 🐼 personlighet besöker just nu.
- Pi : värdet av $P[i]$.
- Denna proceduren anropas exakt $N - 1$ gånger per testfall, i ordningen $i = 1, 2, \dots, N - 1$.

Denna proceduren ska returnera ett heltal $S[i]$ som anger vad Harrys utelåsta 🐼 personlighet ska göra vid ställe i :

- $S[i] = 0$: Harrys utelåsta 🐼 personlighet skickar inte ett brev från ställe i .
- $1 \leq S[i] \leq 20\,000$: Harrys utelåsta 🐼 personlighet skickar heltalet $S[i]$ i ett brev från ställe i .

När Harry kommer hem ska du implementera proceduren för hans inlåsta 🐼 personlighet:

```
std::pair<int,int> longest_path(std::vector<int> S)
```

- S : en lista av längd N sådan att:
 - $S[0] = 0$.
 - För varje $1 \leq i \leq N - 1$, så är $S[i]$ heltalet returnerat av `send_message(N, i, Pi)`.
- Denna proceduren anropas exakt en gång för varje testfall.

Denna proceduren ska returnera ett par ställen (U, V) med största möjliga avstånd.

När koden rättas, så körs ett program som anropar ovanstående funktioner **exakt två gånger**.

- Under första körningen av programmet:
 - `send_message` anropas exakt $N - 1$ gånger.
 - **Ditt program får förvara information mellan på varandra följande anrop.**
 - De returnerade värdena (listan S) kopieras av domarsystemet.
 - I vissa fall är beteendet av domaren **adaptivt**. Detta innebär att värdet av $P[i]$ i ett anrop till `send_message` kan bero på vad Harrys utelåsta 🦉 personlighet har gjort under tidigare anrop.
- Under den andra körningen av ditt program:
 - `longest_path` anropas exakt en gång. Den enda informationen som är tillgänglig till `longest_path` från första körningen är listan S .

Begränsningar

- $N = 10\,000$
- $0 \leq P[i] < i$ för varje i sådan att $1 \leq i \leq N - 1$.

Subtasks och poäng

Subtask	Poäng	Ytterligare begränsningar
1	30	Ställe 0 och något annat ställa har ett största möjliga avstånd bland alla par av ställen.
2	70	Inga ytterligare begränsningar.

Låt Z vara det största heltalet som dyker upp i listan S , och låt M vara det antalet meddelanden som skickas av Harrys utelåsta 🦉 personlighet.

Om något av följande situationer inträffar i något testfall, så kommer din poäng på det testfallet vara 0 (visas som `Output isn't correct` i CMS):

- Åtminstone ett element i S är ogiltigt.

- $Z > 20\,000$ eller $M > 50$.
- Returvärdet för anropet till proceduren `longest_path` är felaktigt.

Annars kommer din poäng för subtask 1 beräknas såhär:

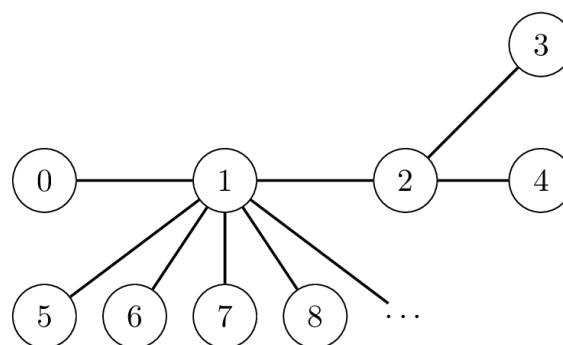
Krav	Poäng
$9\,998 \leq Z \leq 20\,000$	10
$102 \leq Z \leq 9997$	16
$5 \leq Z \leq 101$	23
$Z \leq 4$	30

Din poäng för subtask 2 beräknas såhär:

Krav	Poäng
$5 \leq Z \leq 20\,000$ and $M \leq 50$	$35 - 25 \log_{4000} \left(\frac{Z}{5} \right)$
$Z \leq 4$ and $32 \leq M \leq 50$	40
$Z \leq 4$ and $9 \leq M \leq 31$	$70 - 30 \log_4 \left(\frac{M}{8} \right)$
$Z \leq 4$ and $M \leq 8$	70

Exempelfall

Låt $N = 10\,000$. Betrakta situationen där $P[1] = 0$, $P[2] = 1$, $P[3] = 2$, $P[4] = 2$, och $P[i] = 1$ for $i > 4$.



Föreställ dig att Harrys utelåsta 🦉 personlighetens strategi är att skicka meddelandet $10 \cdot V + U$ varje gång paret av platser (U, V) med det maximala avståndet ändras som efter ett `send_message` anrop.

Ursprungligen är paret med det största avståndet $(U, V) = (0, 0)$. Betrakta följande sekvens av anrop under programmets första körning:

Proceduranrop	(U, V)	Returvärde ($S[i]$)
<code>send_message(10000, 1, 0)</code>	$(0, 1)$	10
<code>send_message(10000, 2, 1)</code>	$(0, 2)$	20
<code>send_message(10000, 3, 2)</code>	$(0, 3)$	30
<code>send_message(10000, 4, 2)</code>	$(0, 3)$	0

Observera att i alla återstående anrop gäller att $P[i] = 1$. Det betyder att paret med det största avståndet inte ändras, och teamet skickar inga fler meddelanden.

Sedan, i programmets andra körning, görs följande anrop:

```
longest_path([0, 10, 20, 30, 0, ...])
```

Harrys inlåsta $\emptyset$ personlighet läser det senaste meddelandet som skickades av forskargruppen, vilket är $S[3] = 30$, och drar slutsatsen att $(0, 3)$ är det par av platser med det största avståndet. Därför returnerar anropet $(0, 3)$.

Observera att denna metod inte alltid gör det möjligt för Harrys inlåsta $\emptyset$ personlighet att korrekt bestämma paret med det maximala avståndet.

Exempeldomare

Exempeldomaren anropar både `send_message` och `longest_path` i samma körning, vilket skiljer sig från den faktiska bedömaren.

Indataformat:

```
N
P[1] P[2] ... P[N-1]
```

Utdataformat:

```
S[1] S[2] ... S[N-1]
U V
```

Observera att du kan använda exempelbetygsättaren med valfritt värde på N .